server with too much work to generate the final hashed value. Users are not going to wait for 10 minutes to log in, while you hash their password twenty million times on underpowered hardware.

Password hashing in Identity is performed by the `PasswordHasher` class. You can configure the number of iterations of the hashing function through the `PasswordHasherOptions` class. The following listing demonstrates how to increase the number of iterations to the number recommended to conform to FIPS.

Listing 13.6 Increasing the `PasswordHasher` PBKDF2 iteration count to 310,000

```
builder.Services.Configure<PasswordHasherOptions>(options =>
{
    options.IterationCount = 310000;
});
```

When the `PasswordHasher` processes a password for storage, it uses a salt—a cryptographically generated random value that is added to the password before it is hashed. This prevents passwords from being cracked through the use of *rainbow tables*—collections of hashes of common passwords. The salt is stored along with the hashed password, the number of iterations that were applied to the hashing function, and the hashing function that was used (SHA256, by default). When a user logs in, the salt and the iteration count are retrieved and then used to hash the submitted password. The result is compared with the stored hash to validate the user.

If the user is validated and the iteration count configured via `PasswordHasherOptions` is higher than the iteration count stored with the password, the password is rehashed using the higher iteration count and updated in the database. This enables you to increase the iteration count over time to keep ahead of technical advances. This operation is not reversible. If you reduce the iteration count, all passwords saved with the higher iteration count will continue to be subject to that higher count when they are verified.

All of this protection provided by Identity is great, but what if you don’t want to use the Identity database schema or UI? Well the good news is that you can still leverage the expert technology within the `PasswordHasher` class itself to ensure you can store and verify passwords securely.

13.3.2 Using the PasswordHasher without Identity UI